

CS 3502 Project 2

CPU Scheduling Simulator

Olamide Akintobi Adedeji
Kennesaw State University

July 24, 2026

1 Introduction

This project evaluates six CPU scheduling algorithms on identical workloads to produce data driven recommendations for OwlTech Industries. I took the hybrid approach permitted by the assignment. The scheduling algorithm logic was implemented as a standalone JavaScript engine and paired with a custom web interface that runs in any browser with no build step or dependencies. The simulator implements the four algorithms from the starter set, which are FCFS, non preemptive SJF, non preemptive Priority, and Round Robin with a configurable quantum. It adds two new algorithms, SRTF and HRRN. All six run on every workload in a single click and report average waiting time, average turnaround time, average response time, CPU utilization, and throughput in a side by side comparison table with Gantt timelines for each algorithm.

The screenshot displays the 'WORKLOAD' section of the simulator. It features four input fields: 'Processes' (30), 'Type' (Mixed), 'Seed' (17), and 'RR quantum' (2). Below these fields are three buttons: 'Generate workload', 'Add process', and 'Clear'. A status message reads: 'Ran 6 algorithms on 30 processes (RR quantum = 2)'. Below the 'WORKLOAD' section is the 'PROCESS TABLE' which lists 12 processes (P1 to P12) with their respective arrival times, burst times, and priorities. Each row also includes a small 'x' icon in the final column.

ID	ARRIVAL	BURST	PRIORITY (1 = HIGHEST)	
P1	24	2	1	x
P2	12	5	3	x
P3	36	5	2	x
P4	29	2	4	x
P5	31	3	4	x
P6	26	4	3	x
P7	40	1	2	x
P8	27	4	1	x
P9	4	5	5	x
P10	33	1	1	x
P11	4	5	4	x
P12	42	5	2	x

Figure 1: The simulator interface with the medium mixed workload loaded. The generator fields fully determine the process set, so every experiment in this report can be regenerated from its seed.

2 Technical Implementation

2.1 Architecture

The implementation separates pure scheduling logic from presentation. The engine file contains the algorithms, the metric calculations, and a seeded workload generator, and has no interface dependencies. The interface file handles the process table, the comparison rendering, the Gantt timelines, and CSV export. This separation is the substance of the hybrid approach since the algorithm core could be attached to a console interface or a test harness without modification. In fact the engine runs unchanged under Node, where a test harness validates every algorithm against hand calculated results before the interface is involved at all.

A design decision worth noting is the shared skeleton for the non preemptive algorithms. FCFS, SJF, Priority, and HRRN are all the same simulation loop. The loop tracks a clock, collects the processes that have arrived, jumps the clock forward over idle gaps, and asks a selection function to choose the next process to run to completion. The four algorithms differ only in that selection function, which makes the comparison between them precise since everything else about the simulation is literally identical code.

2.2 New Algorithm 1, SRTF

Shortest Remaining Time First is the preemptive form of SJF. The implementation advances time one unit at a time. At every unit it selects, from the processes that have arrived and still have work left, the one with the least remaining time, and runs it for exactly one unit. Preemption is not a special case in the code. It emerges naturally, because when a shorter job arrives during a long job's execution, the next decision simply selects the newcomer. Ties break toward the earlier arrival so results are deterministic.

```
1  const p = ready.reduce((a, b) =>
2    (b.remaining < a.remaining ||
3     (b.remaining === a.remaining && b.arrival < a.arrival)) ? b : a);
4  if (p.response === null) p.response = t - p.arrival;
5  schedule.push({ id: p.id, start: t, end: t + 1 });
6  p.remaining -= 1;
7  t += 1;
8  if (p.remaining === 0) { p.completion = t; completed++; }
```

The unit time simulation produces many one unit schedule slices, so a merge pass glues consecutive slices of the same process into readable Gantt bars. A process whose color appears in separate places on the timeline was preempted and later resumed, which makes preemption directly visible in the interface.

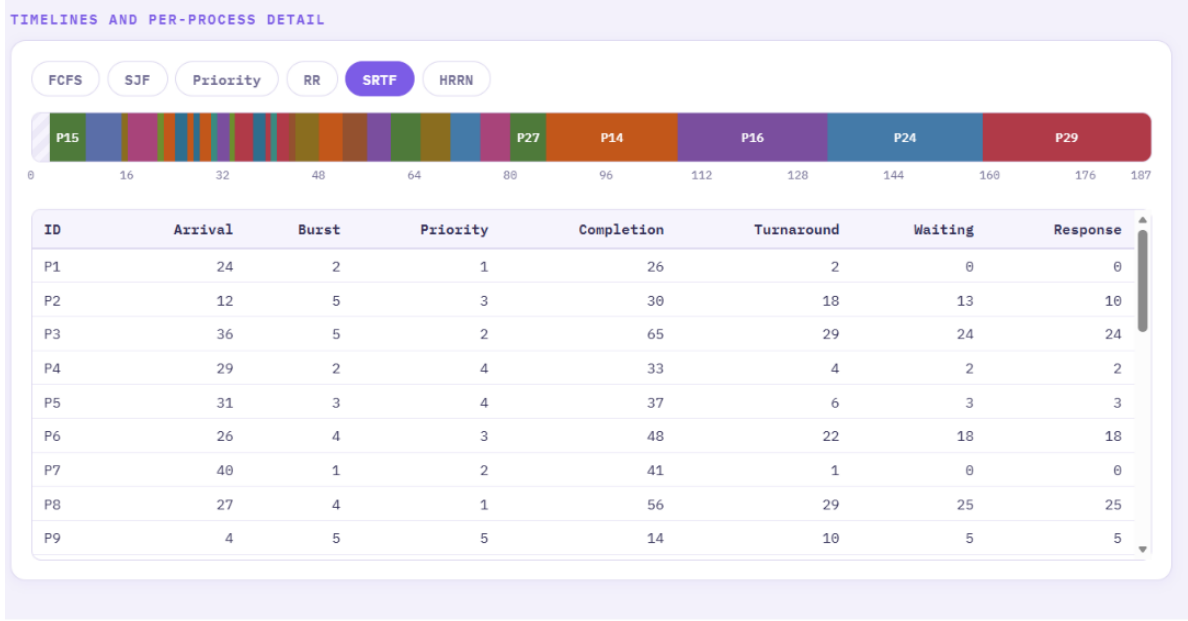


Figure 2: SRTF timeline for the medium mixed workload. Short processes cut in ahead of the long ones, and the long processes P14, P16, P24, and P29 finish last despite arriving in the middle of the run.

2.3 New Algorithm 2, HRRN

Highest Response Ratio Next is non preemptive. At each scheduling decision it computes, for every ready process, the response ratio of waiting time so far plus burst divided by burst, and selects the maximum.

```

1 ready.reduce((a, b) => {
2   const ra = (now - a.arrival + a.burst) / a.burst;
3   const rb = (now - b.arrival + b.burst) / b.burst;
4   return rb > ra ? b : a;
5 });

```

The ratio starts at one for a job that just arrived. A short job's ratio climbs quickly as it waits because the denominator is small, so HRRN keeps most of the efficiency of SJF. A long job's ratio also climbs the longer it is passed over, so it must eventually win the comparison. HRRN therefore exists to fix the starvation problem that SJF and SRTF share, and the performance data later in this report shows how little efficiency it gives up to do so.

2.4 Verification

Correctness was established in three ways. First, all six algorithms were checked against hand calculated schedules for the assignment's four process example, where FCFS must average 8.75 units of waiting and SRTF must average 6.5. Second, a conservation check confirms that every algorithm schedules exactly the total burst time of the workload and stamps a completion time on every process. Third, the small mixed workload from the experiments below was verified by hand. Table 1 shows the FCFS schedule computed manually from the eight process set, and its average waiting time of 17.75 units matches the simulator's reported value exactly. Edge cases from the assignment's testing section are covered by the experiment design itself, including simultaneous

arrivals, identical burst times, one unit bursts mixed with bursts over twenty units, idle gaps before late arrivals, and the starvation scenario that HRRN addresses.

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	2	5	7	5	0
P2	9	4	13	4	0
P3	10	24	63	53	29
P4	6	1	8	2	1
P5	9	26	39	30	4
P6	10	1	64	54	53
P7	6	1	9	3	2
P8	11	1	65	54	53

Table 1: Hand verification of FCFS on the small mixed workload with seed 5. Average waiting is 17.75 units, matching the simulator.

3 Performance Analysis

3.1 Methodology

Nine workloads were tested, covering three sizes and three burst profiles. Small sets have 8 processes, medium sets have 30, and large sets have 150. CPU bound workloads draw bursts between 15 and 40 units, I/O bound workloads draw bursts between 1 and 6 units, and mixed workloads draw 70 percent short bursts and 30 percent long bursts. A simulator without a real I/O subsystem approximates I/O bound behavior as many short CPU bursts, which captures the scheduling relevant property that such processes need the CPU often but briefly. The workload generator uses a seeded deterministic random number generator, so each experiment is fully specified by its process count, type, and seed, and every number in this report can be regenerated. The Round Robin quantum was held at 2 for all nine runs so that differences between runs are attributable to the workload alone.

Run	Type	Processes	Seed
small mixed	Mixed	8	5
small cpu	CPU bound	8	33
small io	I/O bound	8	8
medium mixed	Mixed	30	17
medium cpu	CPU bound	30	2
medium io	I/O bound	30	21
large mixed	Mixed	150	29
large cpu	CPU bound	150	36
large io	I/O bound	150	20

Table 2: Experiment matrix. All runs use RR quantum 2.

3.2 Results

Table 3 reports average waiting time for all nine runs, which is where the algorithms differ most. Table 4 gives the full metric set for the medium mixed run as a representative example.

Run	FCFS	SJF	Priority	RR	SRTF	HRRN
small mixed	17.75	5.50	5.50	9.75	5.00	5.75
small cpu	82.50	64.38	78.63	130.63	64.38	64.50
small io	5.75	4.13	5.50	7.13	3.63	4.63
medium mixed	95.43	24.67	63.80	55.90	24.60	27.10
medium cpu	412.60	316.40	383.43	635.03	309.83	320.40
medium io	35.70	21.57	33.67	44.00	21.33	24.80
large mixed	697.26	212.88	605.41	465.75	212.72	222.37
large cpu	2002.51	1596.85	1883.45	3266.80	1596.34	1608.67
large io	170.49	112.44	170.48	208.80	112.33	127.28

Table 3: Average waiting time in time units across all nine runs. Bold marks the best value per run. SRTF is best in all nine.

Algorithm	Avg waiting	Avg turnaround	Avg response	CPU util %	Throughput
FCFS	95.43	101.57	95.43	98.40	0.1604
SJF	24.67	30.80	24.67	98.40	0.1604
Priority	63.80	69.93	63.80	98.40	0.1604
RR	55.90	62.03	20.13	98.40	0.1604
SRTF	24.60	30.73	24.47	98.40	0.1604
HRRN	27.10	33.23	27.10	98.40	0.1604

Table 4: Full metrics for the medium mixed workload with 30 processes and seed 17.

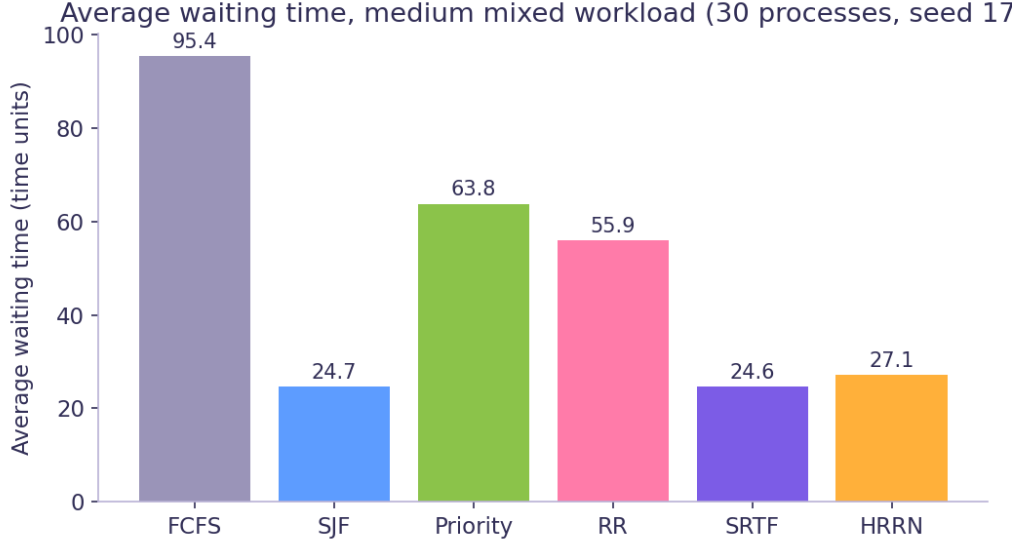


Figure 3: Average waiting time on the medium mixed workload.

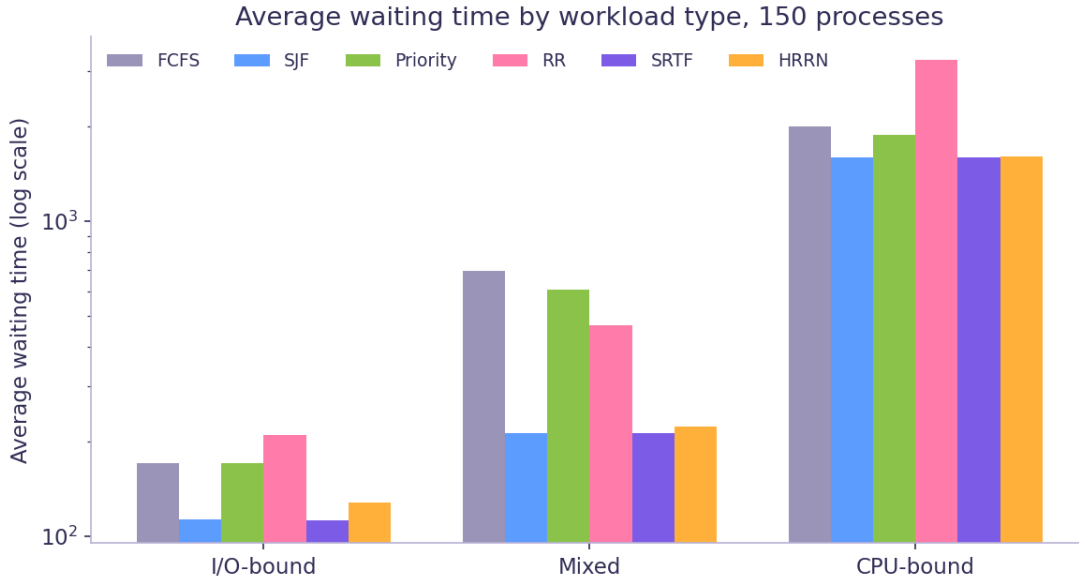


Figure 4: Average waiting time by workload type at 150 processes, log scale. The spread between algorithms is wide on mixed workloads and narrow on CPU bound workloads, where RR becomes the worst performer.

3.3 Discussion

The mixed workloads show the convoy effect clearly. FCFS averaged 95.4 units of waiting on the medium mixed run against 24.7 for SJF, a ratio close to four, and 697 against 213 on the large mixed run. When 70 percent of processes have bursts under seven units and the rest run twenty units or more, arrival order regularly traps a queue of short jobs behind one long one, and each

such event charges waiting time to many processes at once. The effect is consistent across scales, with FCFS to SJF ratios of 3.2, 3.9, and 3.3 on the small, medium, and large mixed runs.

The CPU bound workloads are the counterpoint. When every burst is long, ordering matters much less, and the FCFS to SJF ratio drops to roughly 1.3 at every scale. Round Robin, however, collapses on these workloads. It recorded 635 units of average waiting on the medium CPU bound run against 316 for SJF, and 3267 against 1597 on the large run. A quantum of 2 against bursts of 15 to 40 units slices every job into many pieces, and since there are no short jobs to rescue, the constant preemption stretches every process's finish time without helping anyone.

COMPARISON						
Metric	FCFS	SJF	Priority	RR	SRTF NEW	HRRN NEW
Avg waiting	412.60	316.40	383.43	635.03	309.83	320.40
Avg turnaround	440.27	344.07	411.10	662.70	337.50	348.07
Avg response	412.60	316.40	383.43	27.67	284.23	320.40
CPU util %	99.88	99.88	99.88	99.88	99.88	99.88
Throughput	0.0361	0.0361	0.0361	0.0361	0.0361	0.0361

Shaded cells mark the best value per metric. Lower is better for waiting, turnaround, and response. Higher is better for utilization and throughput.

Figure 5: Simulator comparison for the medium CPU bound workload. RR is worst on waiting and turnaround while best on response, both by wide margins.

The same runs redeem Round Robin on a different metric. RR recorded the best average response time in all nine runs, and the margin is dramatic on CPU bound workloads, with 27.7 units against 316 for SJF on the medium run and 147.5 against 1597 on the large run. Waiting time and response time tell opposite stories about the same algorithm. RR delays completions but gets every process onto the CPU almost immediately, which is exactly the property interactive systems pay for.

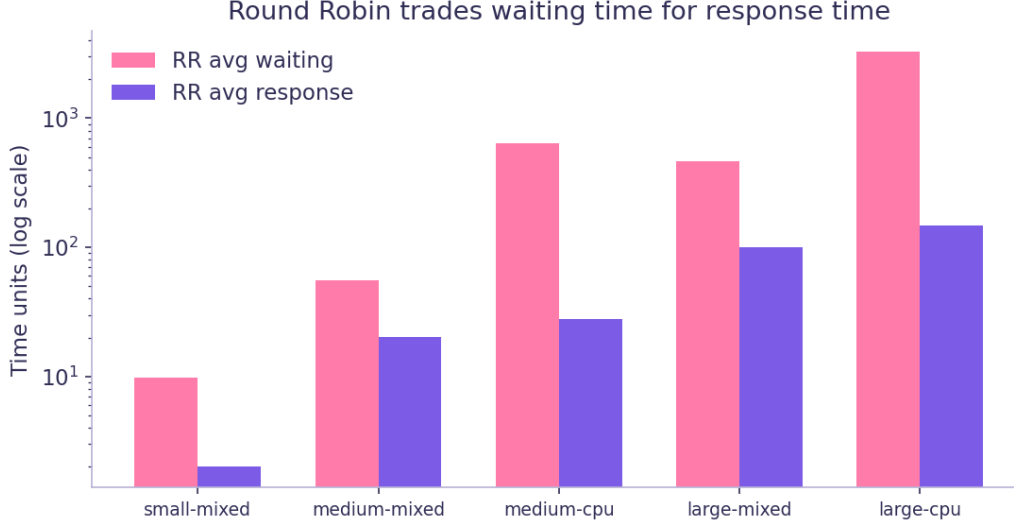


Figure 6: Round Robin average waiting against average response across five runs, log scale. The gap between the bars is the price and the product of time slicing.

SRTF posted the best average waiting time in all nine runs, which follows from theory since SRTF is optimal for average waiting when preemption is allowed. Its margin over non preemptive SJF is small on most runs but visible on the CPU bound sets, where preempting a long burst for a newly arrived shorter one saves real time. HRRN tracked SJF closely everywhere, finishing within about 1 to 13 percent of SJF’s average waiting on every run while remaining starvation proof. Priority scheduling landed between FCFS and SJF on every workload, which is expected since the generated priorities are uncorrelated with burst length. On the large I/O bound run Priority and FCFS tied almost exactly, since with uniformly short bursts the particular ordering matters little.

Utilization and throughput were identical across algorithms within each run and varied only between runs, from 96.3 percent on the small sets to 100 percent on the large ones where no idle gaps remained. This is worth stating plainly since it frames what scheduling can and cannot do. Every algorithm completed the same work in essentially the same span, so the scheduler does not create CPU capacity. What it controls is how waiting is distributed among processes, which is why average waiting varies by a factor of five between algorithms while throughput does not move.

4 Recommendations

The data supports different algorithms for different OwlTech systems rather than a single winner.

For batch and throughput oriented systems where jobs vary in length, SRTF is the strongest choice. It won average waiting on all nine workloads and its advantage is largest exactly where scheduling matters most, on mixed workloads with the convoy risk. Its known weakness is starvation of long jobs under a continuous stream of short arrivals, so it fits best where workloads arrive in batches rather than continuously.

For interactive and user facing systems, Round Robin is the right tool despite its poor waiting times. It delivered first CPU contact an order of magnitude faster than every alternative on the heavy workloads, and responsiveness is what users perceive. Its quantum must be tuned to the workload, since the CPU bound results show a small quantum taxing long jobs severely. Raising

the quantum moves RR toward FCFS behavior, so the value should sit just above the burst length of typical interactive requests.

As a general purpose default, HRRN is the recommendation. It captured nearly all of SJF's efficiency on every workload while carrying a structural guarantee against starvation, which SJF and SRTF lack. A system that cannot predict its workload mix gives up very little by choosing HRRN and gains predictable fairness.

Plain FCFS should be avoided wherever burst lengths vary, since the convoy effect measured here is severe and grows with load. Priority scheduling is only as good as the meaning of its priorities, and if used it should be combined with aging so that low priority processes inherit the same protection HRRN provides.

Future work could add I/O burst modeling with blocked states, an MLFQ implementation to combine RR's responsiveness with better turnaround, and a quantum sensitivity study extending the CPU bound observation above.

5 Conclusion

Six scheduling algorithms ran on nine reproducible workloads spanning three sizes and three burst profiles. The measurements matched theory in every case. FCFS suffers the convoy effect on varied workloads, SRTF is unbeaten on average waiting, Round Robin trades completion speed for response speed, and HRRN buys starvation protection for a few percent of SJF's efficiency. The practical conclusion is that scheduling policy should follow the workload, and the simulator built for this project makes that comparison a one click experiment.

References

- [1] J. Bell, Operating Systems Course Notes, Chapter 5, CPU Scheduling. University of Illinois Chicago. https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/5_CPU_Scheduling.html
- [2] R. Arpaci-Dusseau and A. Arpaci-Dusseau, Operating Systems, Three Easy Pieces. Arpaci-Dusseau Books, 2018. Chapters 7 and 8.